

Mini-Issue LAB 1

目标

实现一个最小用户系统。

需要完成：

- 用户注册
- 用户登录
- JWT 鉴权
- 获取当前用户信息

技术栈：

- Go
- Gin
- sqlx
- MySQL
- JWT
- bcrypt

一、项目结构

创建如下目录：

```
mini-issue/
├── cmd/
│   └── main.go
├── internal/
│   ├── model/
│   │   └── user.go
│   ├── dao/
│   │   └── user_dao.go
│   ├── service/
│   │   └── user_service.go
│   ├── controller/
│   │   └── user_controller.go
│   ├── middleware/
│   │   └── auth.go
│   └── router/
│       └── router.go
├── pkg/
│   ├── db/
│   │   └── mysql.go
│   └── jwt/
```

```
| | | jwt.go
| | | response/
| | | response.go
| | go.mod
```

二、数据库

创建数据库

数据库名：

mini_issue

创建 users 表

字段：

字段名	类型	说明
id	bigint	自增主键
username	varchar(64)	用户名
password	varchar(255)	bcrypt 密码
created_at	timestamp	创建时间

约束：

- username 唯一
- username 非空
- password 非空

三、分层架构

本项目采用：

```
Controller
↓
Service
↓
DAO
```

↓
MySQL

四、各层职责

1. Controller

负责：

- HTTP 请求
- JSON 绑定
- 返回 JSON 响应

不负责：

- SQL
 - JWT 生成
 - 密码校验
 - bcrypt
-

2. Service

负责：

- 业务逻辑
- 参数校验
- 密码加密
- JWT 生成
- 用户校验

不负责：

- HTTP
 - SQL
-

3. DAO

负责：

- SQL 查询
- SQL 插入
- 数据库交互

不负责：

- HTTP
- JWT
- 密码校验
- 业务逻辑

五、数据结构设计

文件：

```
internal/model/user.go
```

需要定义：

User

字段：

字段	类型
ID	int64
Username	string
Password	string
CreatedAt	time.Time

要求：

- 支持 sqlx 的 db tag
- password 不能返回给前端

RegisterRequest

字段：

字段	类型
Username	string
Password	string

LoginRequest

字段：

字段	类型
Username	string
Password	string

LoginResponse

字段：

字段	类型
Token	string

六、DAO 设计

文件：

```
internal/dao/user_dao.go
```

UserDAO

需要保存：

```
db
```

类型：

```
*sqlx.DB
```

NewUserDAO

作用：

创建 DAO 对象。

参数:

```
db *sqlx.DB
```

返回:

```
*UserDAO
```

CreateUser

作用:

向 users 表插入用户。

参数:

参数	类型
username	string
hashedPassword	string

说明:

- password 必须已经是 bcrypt 加密后的结果
- DAO 不负责加密

返回:

```
error
```

成功:

```
nil
```

失败:

返回数据库错误。

GetByUsername

作用:

根据用户名查询用户。

参数：

参数	类型
username	string

返回：

```
(*User, error)
```

三种情况：

1. 查到用户

```
return &user, nil
```

2. 用户不存在

```
return nil, nil
```

3. SQL 执行失败

```
return nil, err
```

GetByID

作用：

根据用户 ID 查询用户。

参数：

参数	类型
id	int64

返回：

```
(*User, error)
```

规则与 GetByUsername 相同。

七、JWT 模块

文件：

```
pkg/jwt/jwt.go
```

Claims

至少包含：

字段
UserID
Username
Exp

GenerateToken

作用：

生成 JWT。

参数：

参数
userID
username

返回：

```
(token string, err error)
```


要求：

- 使用 HS256
- 设置过期时间

ParseToken

作用：

解析 JWT。

参数：

token string

返回：

(*Claims, error)

要求：

- 校验 token 是否合法
- 校验是否过期

八、Service 设计

文件：

internal/service/user_service.go

UserService

保存：

userDAO

Register

作用：

处理注册逻辑。

流程：

1. 校验用户名不能为空
2. 校验密码长度 ≥ 6
3. 查询用户名是否存在
4. bcrypt 加密密码
5. 调 DAO 创建用户

参数：

RegisterRequest

返回：

error

Login

作用：

处理登录逻辑。

流程：

1. 校验参数
2. 根据 username 查用户
3. 校验密码
4. 生成 JWT
5. 返回 token

返回：

(*LoginResponse, error)

密码校验必须使用：

```
bcrypt.CompareHashAndPassword
```

GetMe

作用：

获取当前用户信息。

参数：

```
userID
```

流程：

1. 根据 ID 查询用户
2. 用户不存在则报错
3. 返回用户信息

返回：

```
(*User, error)
```

九、Middleware 设计

文件：

```
internal/middleware/auth.go
```

AuthMiddleware

作用：

校验 JWT。

流程：

1. 读取 Authorization Header
2. 校验 Bearer 格式

3. 解析 JWT
4. 校验 token
5. 将 user_id 写入 gin.Context

要求：

context 至少保存：

```
user_id  
username
```

失败时：

返回 401。

十、Controller 设计

文件：

```
internal/controller/user_controller.go
```

Register

流程：

1. 绑定 JSON
2. 调 service.Register
3. 返回统一 JSON

Login

流程：

1. 绑定 JSON
2. 调 service.Login
3. 返回 token

Me

流程：

1. 从 Context 获取 user_id
2. 调 service.GetMe
3. 返回当前用户

十一、Router

文件：

```
internal/router/router.go
```

注册接口

公开接口：

```
POST /register  
POST /login
```

需要鉴权：

```
GET /me
```

十二、统一响应格式

文件：

```
pkg/response/response.go
```

Success

返回：

```
{
  "code": 0,
  "msg": "success",
  "data": {}
}
```

Fail

返回：

```
{
  "code": -1,
  "msg": "错误信息",
  "data": null
}
```

十三、main.go

只负责：

1. 初始化数据库
2. 创建 DAO
3. 创建 Service
4. 创建 Controller
5. 创建 Router
6. 启动服务

不要在 main.go 写业务逻辑。

十四、接口验收

你必须通过：

注册成功

```
POST /register
```

新用户能成功创建。

重复用户名失败

重复 username 必须失败。

密码过短失败

密码长度 < 6 必须失败。

登录成功

正确账号密码：

POST /login

返回 JWT。

登录失败

错误密码必须失败。

/me 未登录失败

不带 token：

GET /me

返回 401。

/me 登录成功

带合法 token：

返回当前用户信息。

password 不得返回

任何接口：

password 不能出现在 JSON 响应中